Nithin Chunchu

Rollno:2303A53032

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE	**DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**
Program Name: B. Tech	**Assignment Type: Lab** **Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Rapelly Nandini Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju

CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week1 - Wednesday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches

Assignment Number:1.3(Present assignment number)/**24**(Total number of assignments)

Q.No.	**Question**	***Expected Time to complete***
1	*Understanding AI-assisted Coding Workflow* Lab 1: Envi ronment Setup – *Gi tHu b Co p ilot and VS Co de In tegration +*	Week1 - Monday

Lab Objectives:

- To install and config ure GitHub C opi l ot i n Visual Studio C ode.
- To expl ore A I -assi sted code generation usi ng GitHub Copi l ot.
- To anal yze the ac curacy and effectiveness of Copilot's code suggestions.
- To understand prom pt-based programm i ng usi ng comments and code con text

Lab Ou tcomes (LOs):
A fter compl eting this l ab, students wi l l be abl e to:

- Set up GitHub C opi l ot i n VS C ode successful l y .
- U se i nline comments and context to generate c ode with C opi l ot.
- Eval uate AI -g enerated code for correctness and reada bi l i ty.
- C om pare code sugg estions based on di fferent prompts and program mi ng styles.

Task 0

- I nstall and config ure GitHub Copilot i n VS C ode. Take screenshots of each step. Expected Ou tput
- I nstall and config ure GitHub Copilot i n VS C ode. Take screenshots of each step.

Task 1: AI -Generated Log i c Wi thout Modul ari zation (Fi bonacci Sequence Wi thout Func tions)

❖ **S cen a rio**
You are asked to write a qui ck numeri cal sequence generator for a learni ng pl atform prototype.

❖ **T a sk Description**
U se GitHub C opil ot to generate a Python prog ram tha t :
- ➢ Prints the Fibonacci sequence up to *n* terms

	➢ A ccepts user i nput f or *n*	

- Implements the logic directly in the main code
- Does not use any user-defined functions

❖ **Expected Output**

- Correct Fibonacci sequence for given *n*
- Screenshot(s) showing Copilot-generated suggestions
- Sample inputs and outputs

Solution:

Write a python program that takes an integer n as input prints the Fibonacci sequence upto to n terms.

#code

```
def fibonacci(n):
    a, b = 0, 1
    for _ in range(n):
        print(a, end=' ')
        a, b = b, a + b
n = int(input("Enter the number of terms: "))
fibonacci(n)
```

#Output

```
Enter the number of terms: 10
0 1 1 2 3 5 8 13 21 34
```

Task 2: AI Code Optimization & Cleanup (Improving Efficiency)

❖ **Scenario**

The prototype will be shared with other developers and needs optimization.

❖ **Task Description**

- Examine the Copilot-generated code from Task 1 and improve it by:
- Removing redundant variables
- Simplifying loop logic
- Avoiding unnecessary computations
- Use Copilot prompts such as:
  - *"Optimize this Fibonacci code"*
  - *"Simplify variable usage"*

Hint:

Prompt Copilot with phrases like
"optimize this code", *"simplify logic"*, or *"make it more readable"*

❖ **Expected Output**
- ➢ Original vs improved code
- ➢ Written explanation of:
  - ▪ What was inefficient
  - ▪ How the optimized version improves performance and readability

Solution:

Write an optimized Python program that takes an integer n as input and prints the Fibonacci sequence up to n terms by simplifying the logic and removing unnecessary variables, without using any user-defined functions.

#Code

```
n = int(input("Enter the number of terms: "))
a, b = 0, 1
for _ in range(n):
    print(a, end=' ')
    a, b = b, a + b
```

#Output

```
Enter the number of terms: 5
0 1 1 2 3
```

Task 3: Modular Design Using AI Assistance (Fibonacci Using Functions)

❖ **Scenario**

The Fibonacci logic is now required in multiple modules of an application.

❖ **Task Description**

Use GitHub Copilot to generate a function-based Python program that:
- ➢ Uses a user-defined function to generate Fibonacci numbers
- ➢ Returns or prints the sequence up to *n*
- ➢ Includes meaningful comments (AI-assisted)

❖ **Expected Output**
- ➢ Correct function-based Fibonacci implementation
- ➢ Screenshots documenting Copilot's function generation
- ➢ Sample test cases with outputs

Solution:

Write a Python program that uses a user-defined function to generate and print the Fibonacci sequence up to n terms, accepts user input for n, and

includes clear, meaningful comments.

```
#Code

# Function to print Fibonacci series
def fibonacci(n):
    a, b = 0, 1  # First two numbers

    for _ in range(n):   # Loop n times
        print(a, end=' ')
        a, b = b, a + b  # Update values

n = int(input("Enter the number of terms: "))  # User input
fibonacci(n)  # Function call

#Output
```

```
Enter the number of terms: 12
0 1 1 2 3 5 8 13 21 34 55 89
```

Task 4: Comparative Analysis – Procedural vs Modular Fibonacci Code

- **Scenario**
  You are participating in a code review session.

- **Task Description**
  Compare the Copilot-generated Fibonacci programs:
  - Without functions (Task 1)
  - With functions (Task 3)
  - Analyze them in terms of:
    - Code clarity
    - Reusability
    - Debugging ease
    - Suitability for larger systems

- **Expected Output**
  Comparison table or short analytical report

Solution:

Write a Python program to generate the Fibonacci series for n terms, once without using functions and once using functions.

#Code

Without using functions

```
n = int(input("Enter the number of terms: "))
a, b = 0, 1
for _ in range(n):
   print(a, end=' ')
   a, b = b, a + b
print()
```

Using functions

```
def fibonacci(n):
   a, b = 0, 1
   for _ in range(n):
      print(a, end=' ')
      a, b = b, a + b
```

#Output

```
Enter the number of terms: 3
0 1 1
```

Task 5: AI-Generated Iterative vs Recursive Fibonacci Approaches (Different Algorithmic Approaches for Fibonacci Series)

- **Scenario**
  Your mentor wants to assess AI's understanding of different algorithmic paradigms.

- **Task Description**
  Prompt GitHub Copilot to generate:
  An iterative Fibonacci implementation
  A recursive Fibonacci implementation

- **Expected Output**
  - Two correct implementations
  - Explanation of execution flow for both

- C ompari son coveri ng :
  - Time and space compl exi ty
  - P erformance for l arg e *n*
  - When recursion shoul d be avoi ded

Solution:

W rite Pyth on programs f o r the Fibon acci series using ite rative a nd recursive metho ds, explain how ea ch works, and comp are th em in terms of time, space, p erf o rma nce f o r large n, and when recursion should be a vo ided

#Cod e

```
# Iterative meth od

d ef f ibonacci_iterative(n):
 a , b = 0, 1 f o r _
 in range(n): p
 rint(a, end=' ') a ,
 b = b, a + b
 pr int()


#Recurs ive method

d ef f ibon acci_recursive(n, a=0, b=1):
 if n > 0:
   p rint(a, end=' ')
   f ibona cci_recursive(n - 1, b, a + b)
 else:
   print()
```

#Ou tpu t

```
Enter the number of terms: 12
0 1 1 2 3 5 8 13 21 34 55 89
```